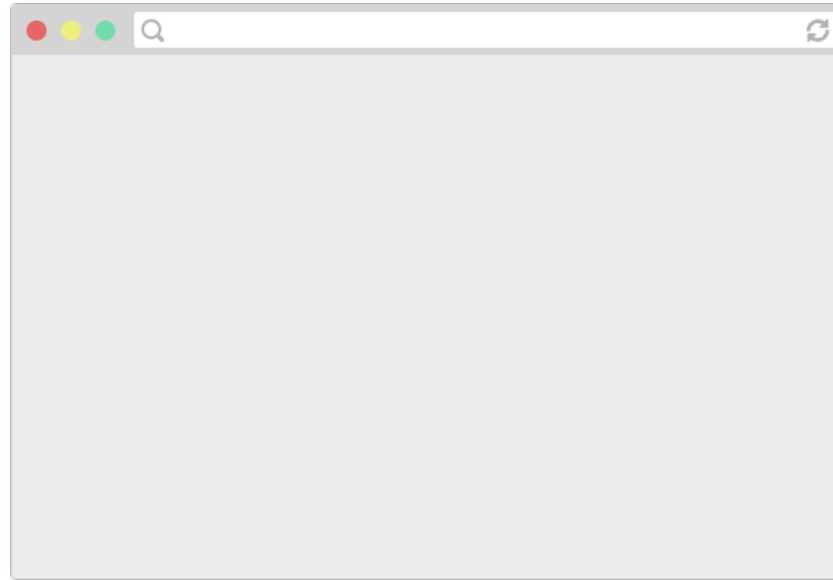


Web Foundations

DM585 Distributed and Web Programming

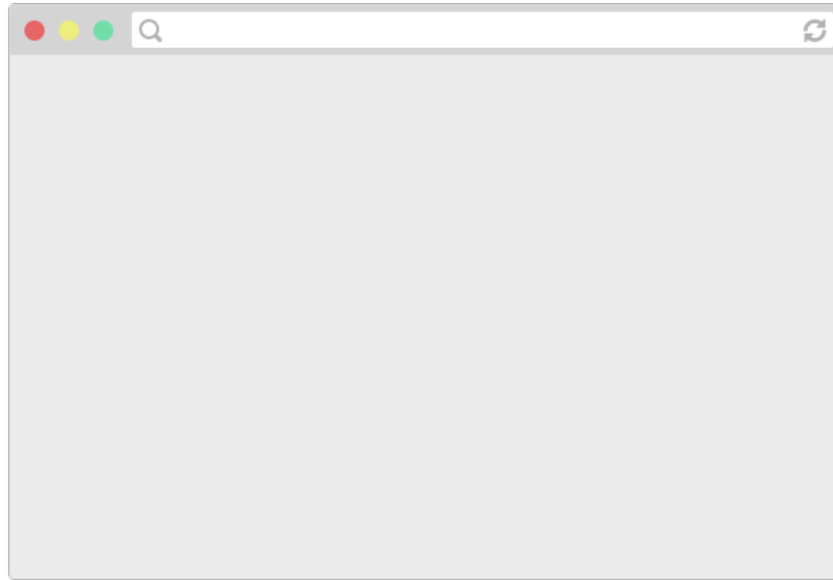
How do web pages work?

How do web pages work?



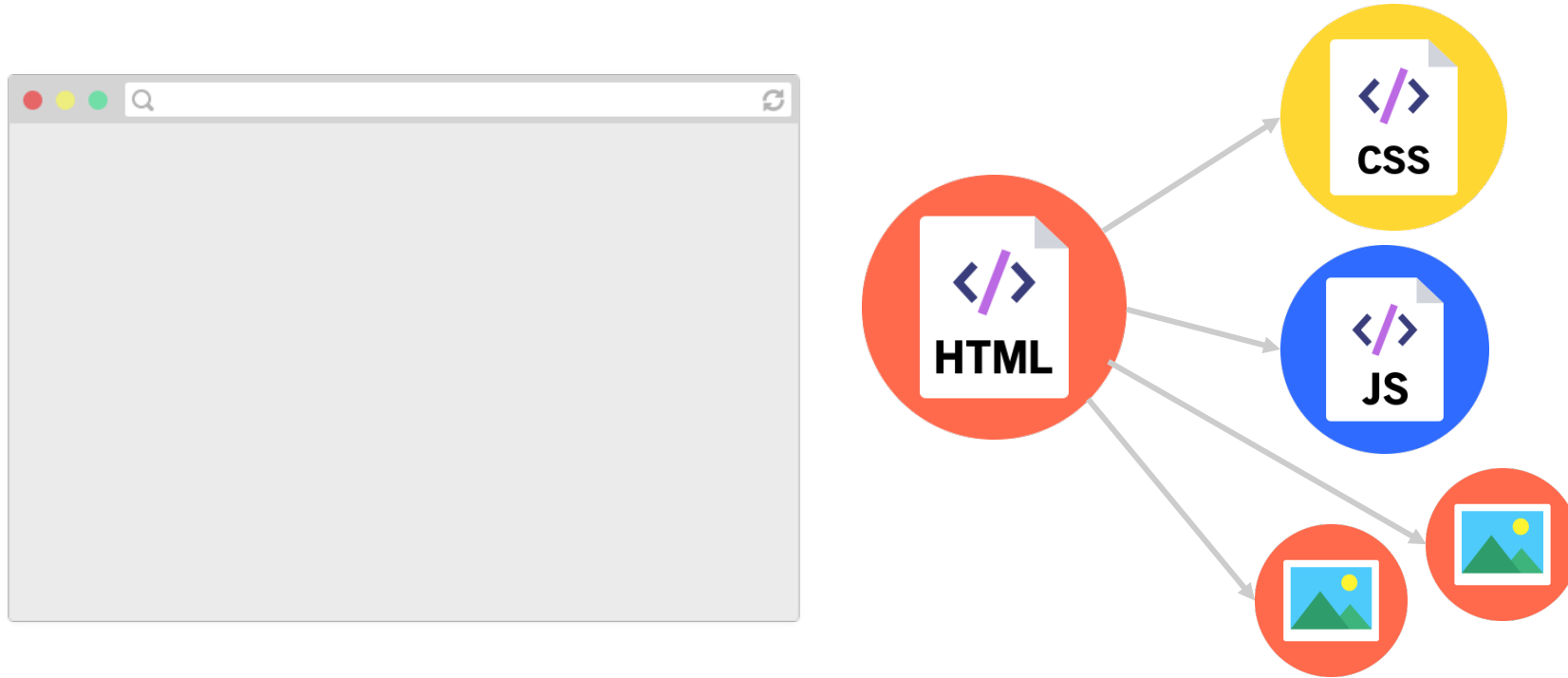
Browsers are applications that can display web pages.
E.g. Firefox, Brave, Vivaldi, Opera, Chrome, Safari, Edge, etc.

How do web pages work?



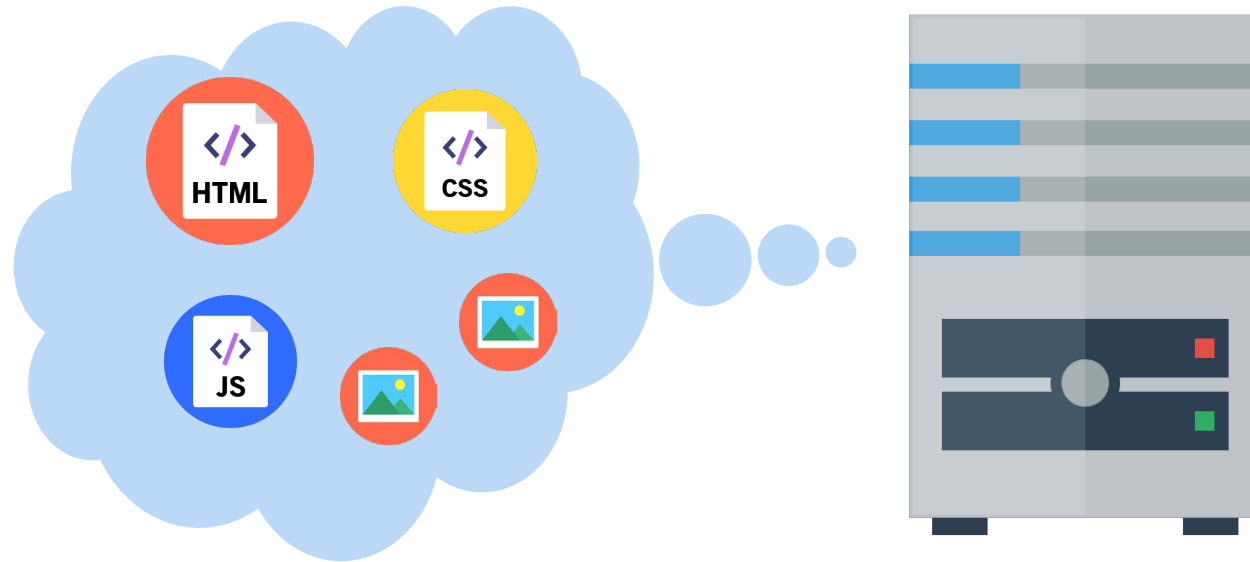
Web pages are written in a markup language called **HTML**, so browsers display a web page by reading and interpreting its HTML.

How do web pages work?



The HTML file might link to other resources, like images, videos, as well as **JavaScript** and **CSS** (stylesheet) files, which the browser then also loads.

How do web pages work?

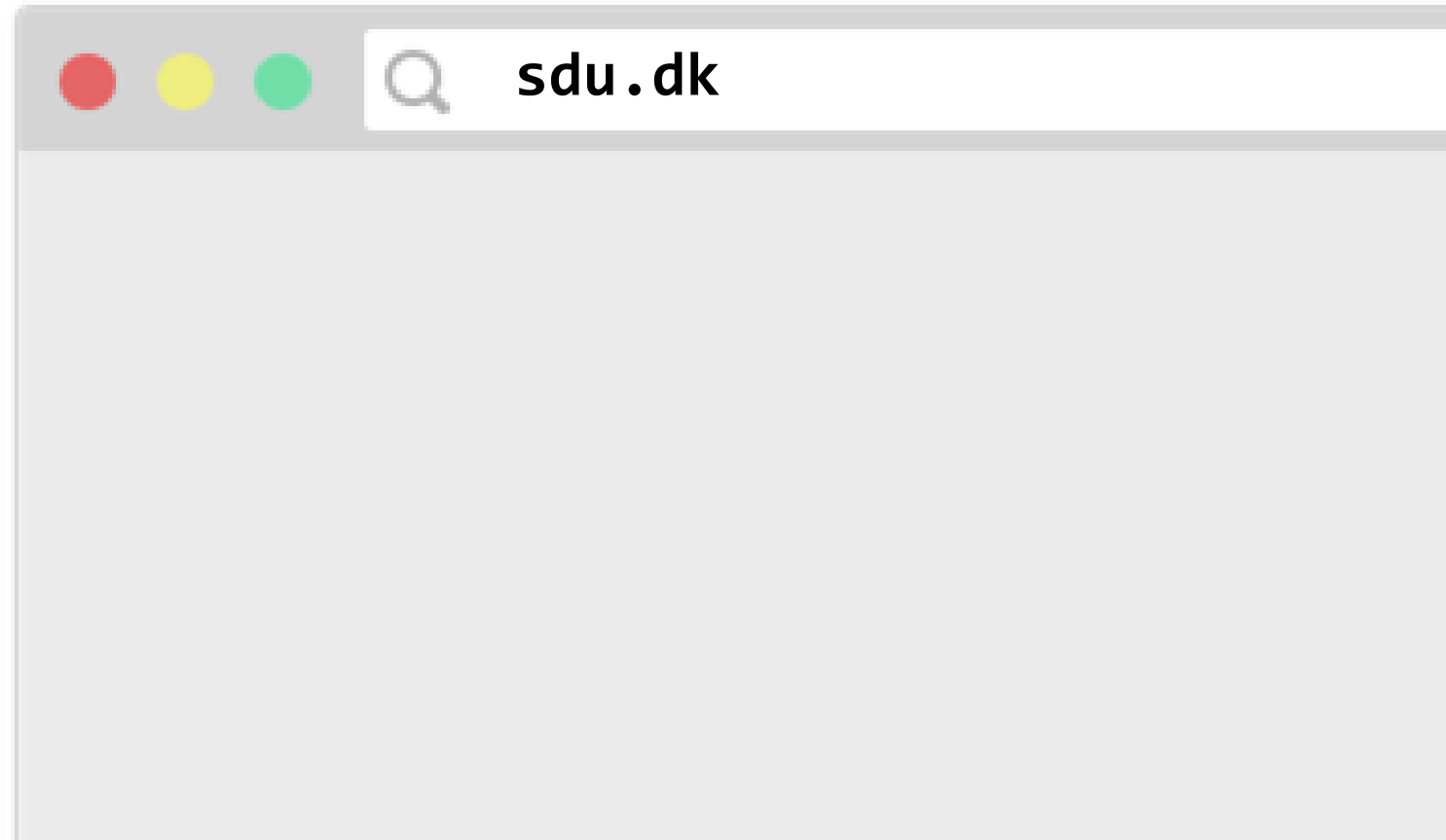


A **web server** is a program running on a computer that delivers web pages in response to requests.

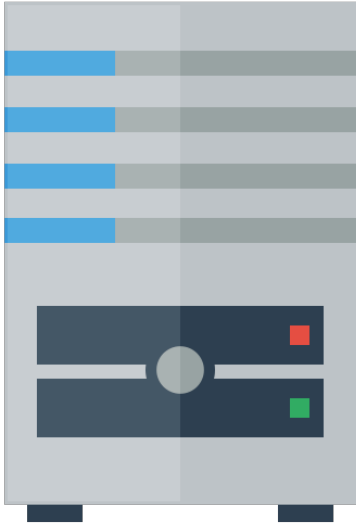
It either stores or generates the web page returned.

How do web pages work?

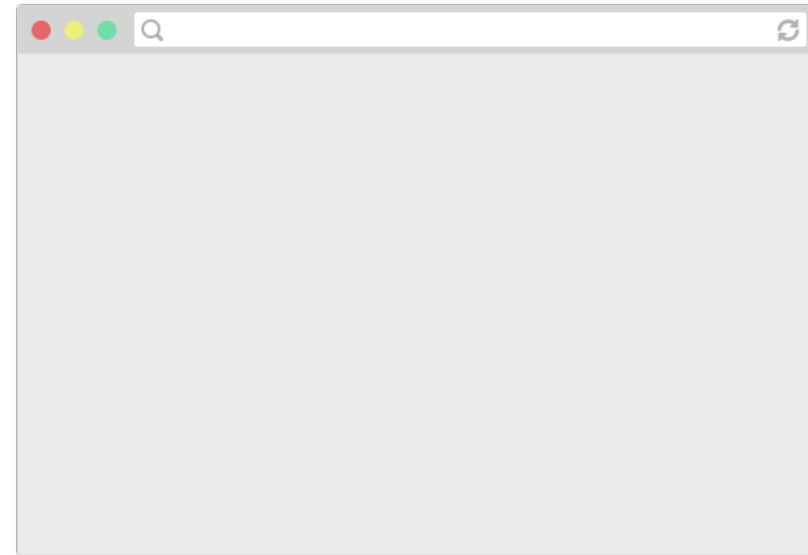
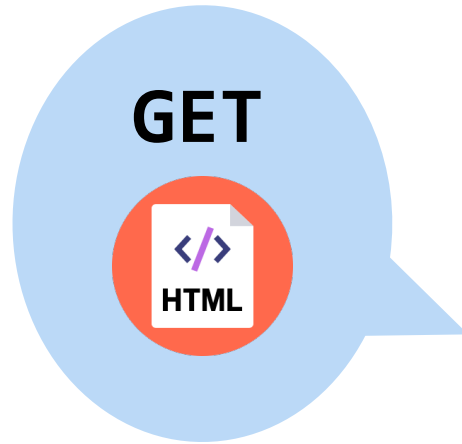
1. You type
in a URL,
which is the
address of
the HTML
file on the
internet.



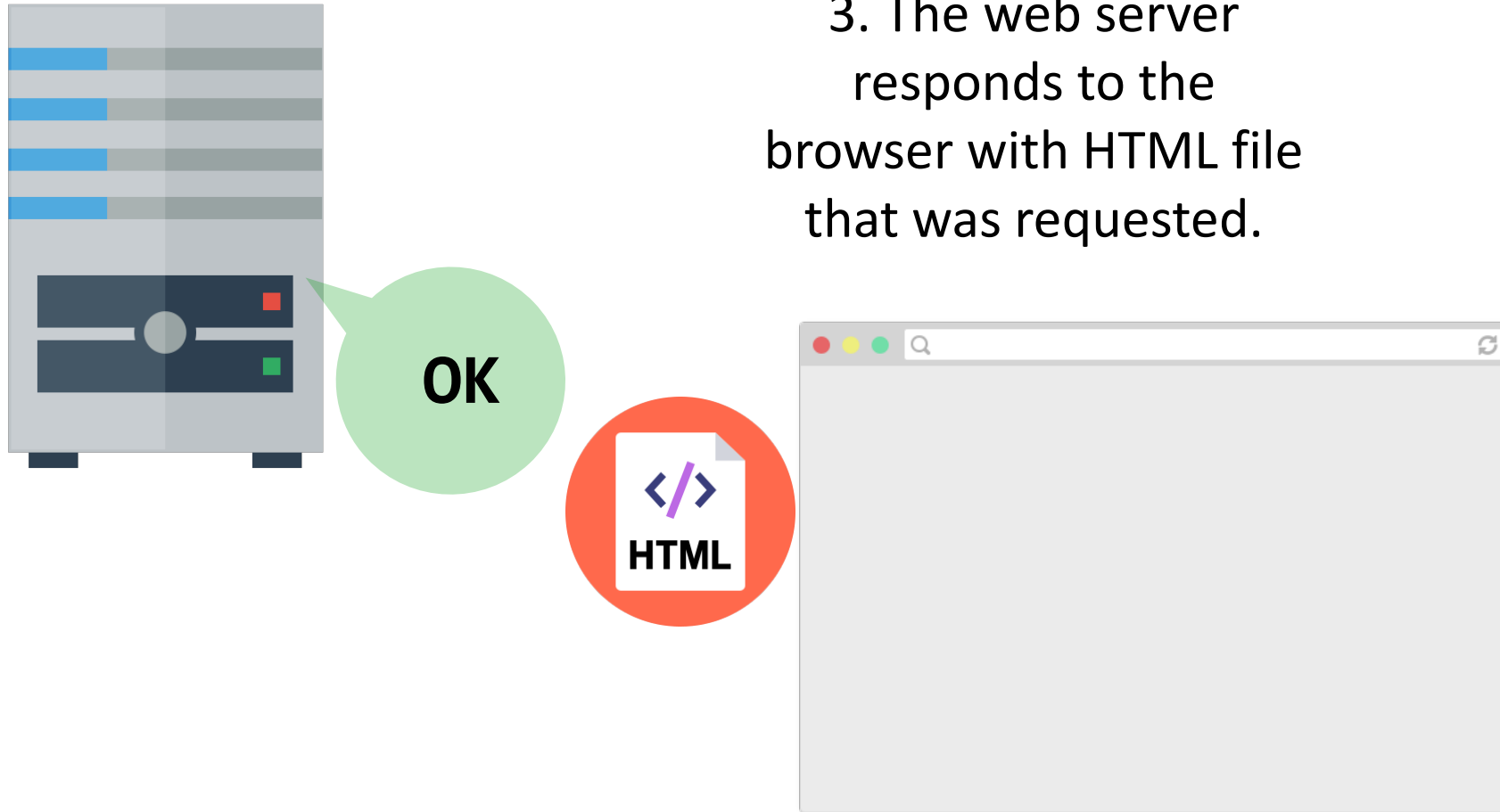
How do web pages work?



2. The browser asks the web server that hosts the document to send that document.

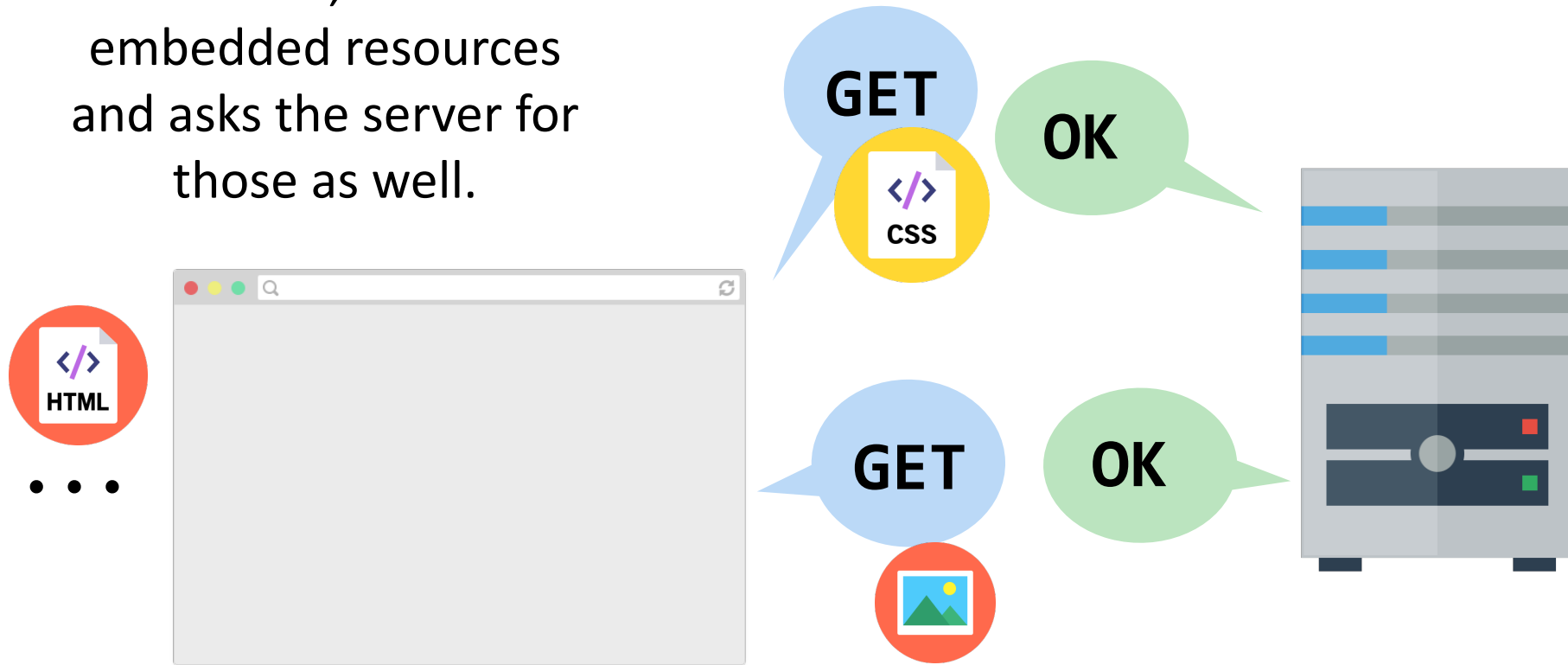


How do web pages work?



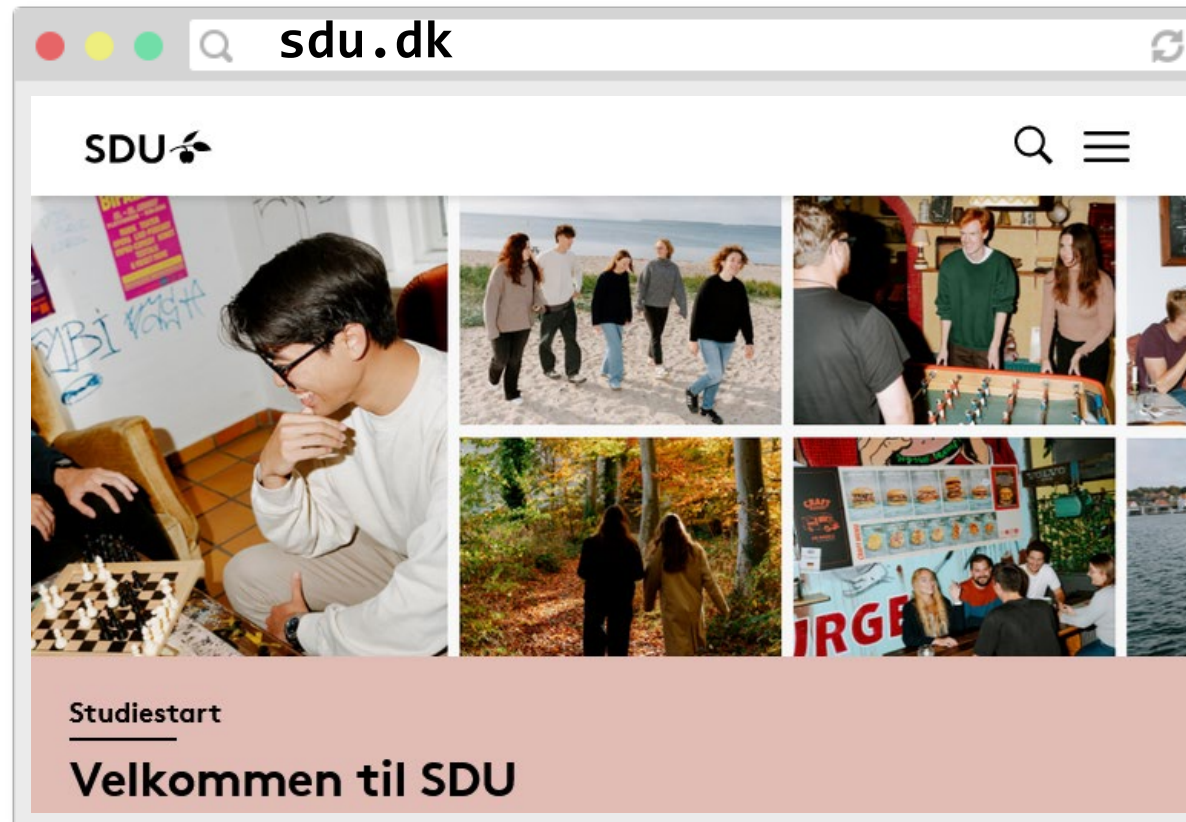
How do web pages work?

4. The browser reads the HTML, sees the embedded resources and asks the server for those as well.



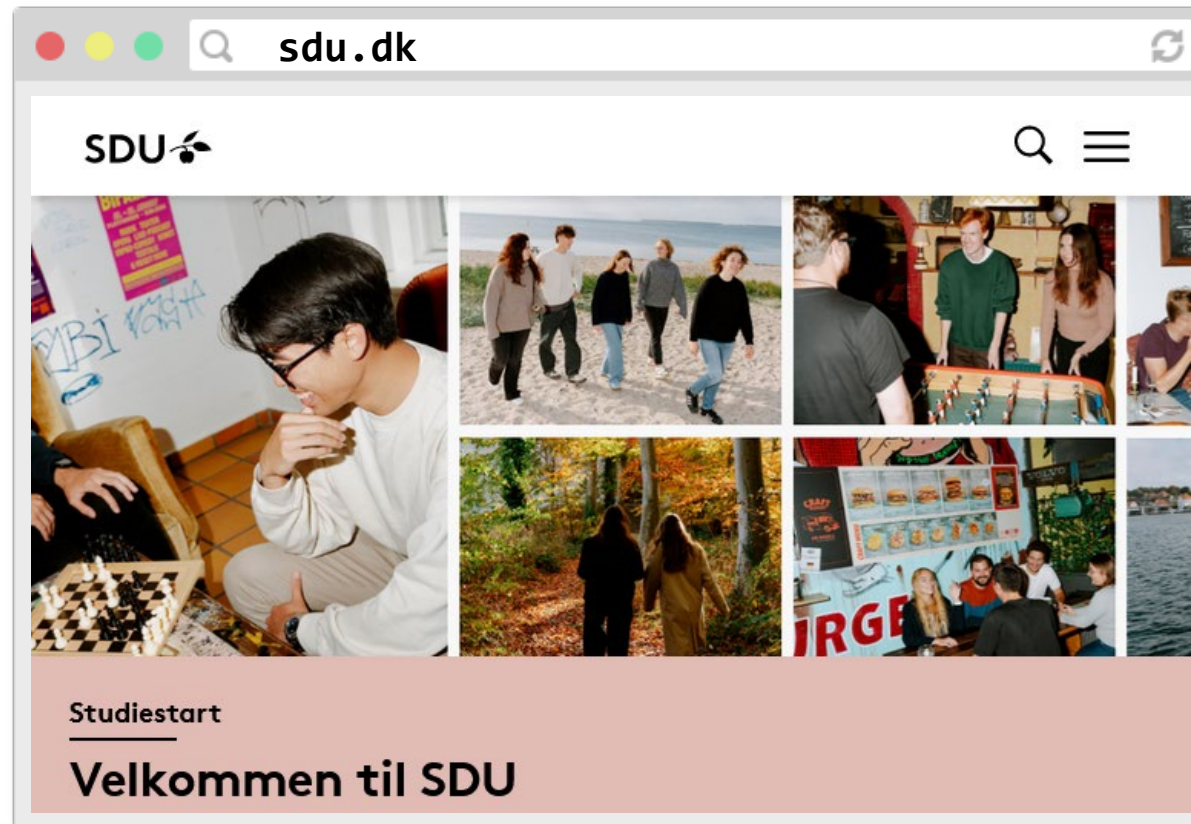
How do web pages work?

5. The web page is loaded when all the resources are fetched and displayed.

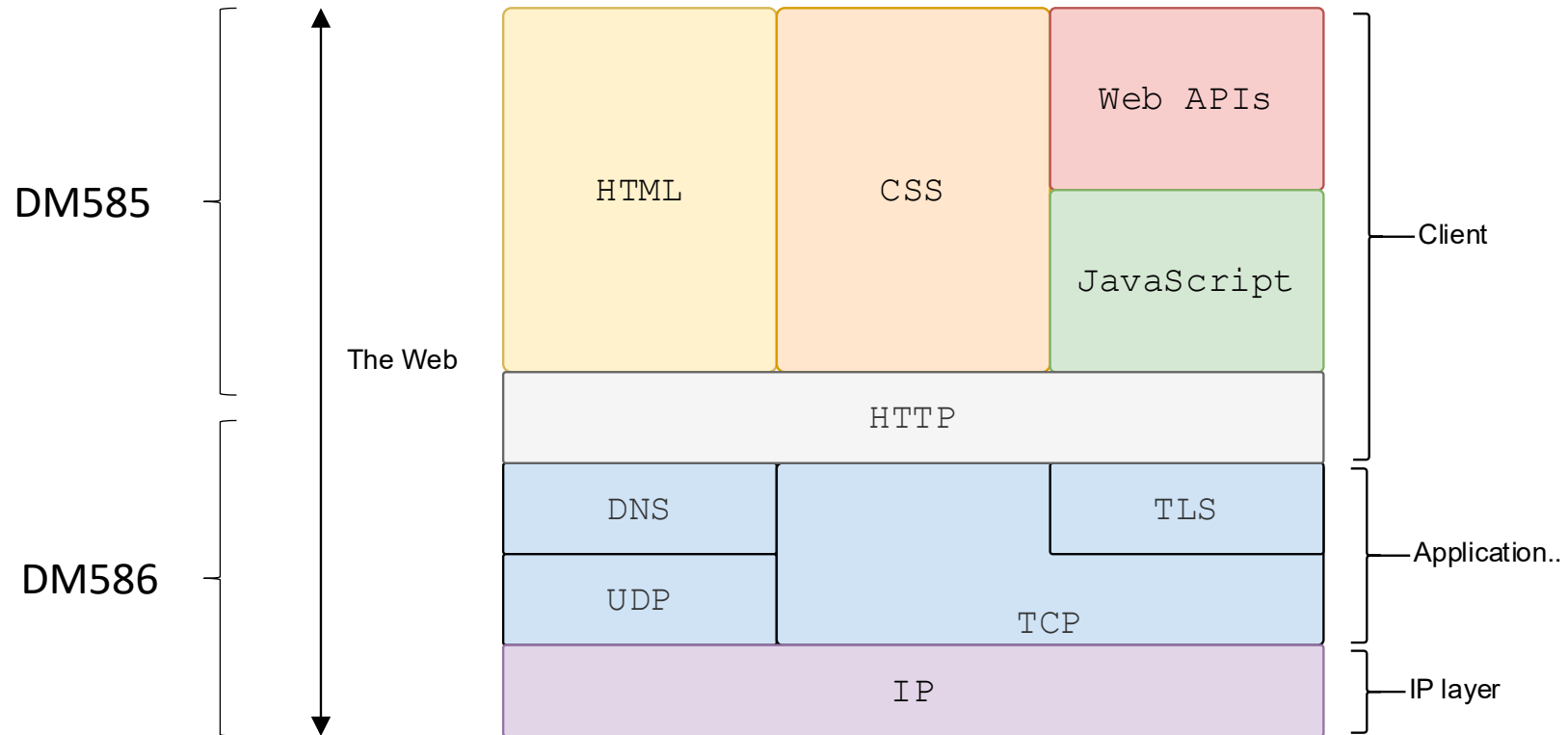


How do web pages work?

(That was obviously very hand-wavy. We'll get more detailed when we talk about servers later in the course.)



The Web (Stack)



What are web pages?

A quick introduction to HTML & CSS

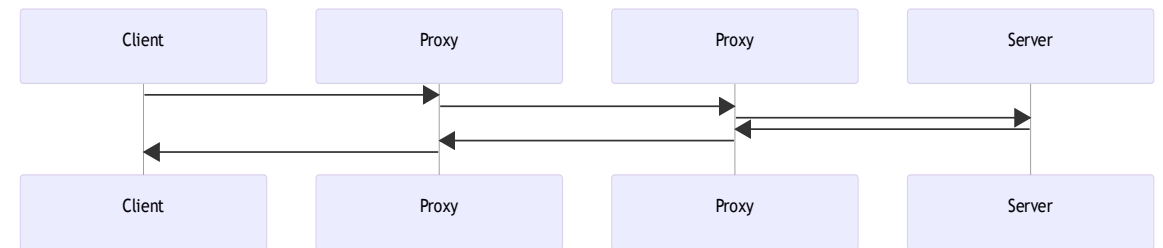
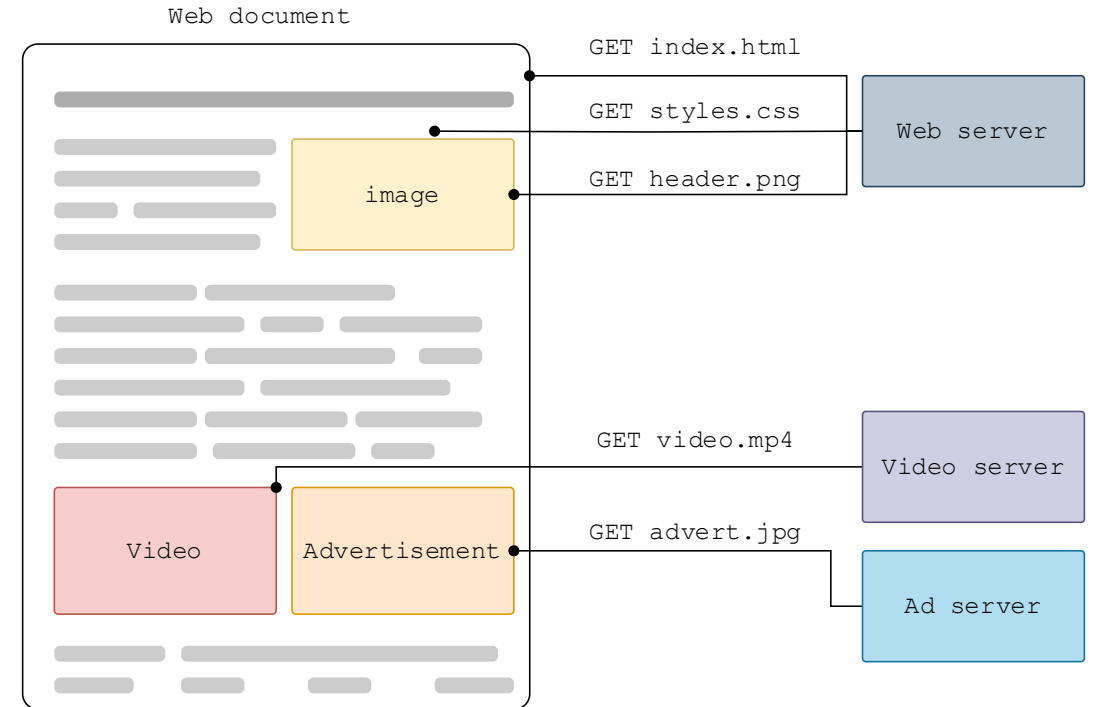
HyperText

- Non-linear text that contains references (called hyperlinks or simply links) to other text.
- Hypertext documents may also reference or embed other media (images, videos, audio, etc).
- Web pages are a prototypical example.
- Hypertext/media is a core concept of the web



HyperText Transfer Protocol (HTTP)

- Application layer protocol
- Designed for web documents/hypertext
 - interactions are directed to the resources fetch/modify
 - interactions are based on single request-response messages
- Client-server model
 - Client (user-agent) initiates interactions by sending a request to a web server
 - Server awaits requests and serves resources accordingly
 - Interactions may be layered with intermediate proxies that offer e.g., caching, filtering, load balancing, logging, authentication.



We will delve deeper into HTTP later in this course (and DM586)

HTML and CSS strategy

Assumption:

- Most people have cursory familiarity with HTML and CSS.

Therefore, we will:

- Speed through the obvious stuff
- Skip self-explanatory syntax
- Skip the parts you can look up easily through a quick web search

Therefore, you should:

- Ask questions without hesitation!

What is HTML?

HTML (HyperText Markup Language)

- Describes the content and structure of a web page; not a programming language.
- Made up of building blocks called **elements**.

`<p>`

HTML is `<em>`awesome!!!`</em>`

``

`</p>`

Basic HTML page structure

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello HTML</title>
  </head>

  <body>
    ... contents of the page...
  </body>
</html>
```

(saved in a *filename.html* file)

Basic HTML page structure

Metadata that doesn't appear
in the viewport of the browser
(the “page” area)

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello HTML</title>
  </head>

  <body>
    ... contents of the page...
  </body>
</html>
```

(saved in a *filename.html* file)

Basic HTML page structure

Metadata that doesn't appear
in the viewport of the browser
(the “page” area)

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello HTML</title>
  </head>
```

Contents that render in the
viewport of the browser

```
  <body>
    ... contents of the page ...
  </body>
</html>
```

(saved in a *filename.html* file)

Basic HTML page structure

Metadata that doesn't appear in the viewport of the browser (the “page” area)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Hello HTML</title>
```

```
</head>
```

E.g. **<title>** defines the page title, and it shows up as the name of the tab

Contents that render in the viewport of the browser

```
<body>
```

```
... contents of the page...
```

```
</body>
```

```
</html>
```

(saved in a *filename.html* file)

HTML elements

`<p>`

HTML is `<em>awesome!!!</em>`

``

`</p>`

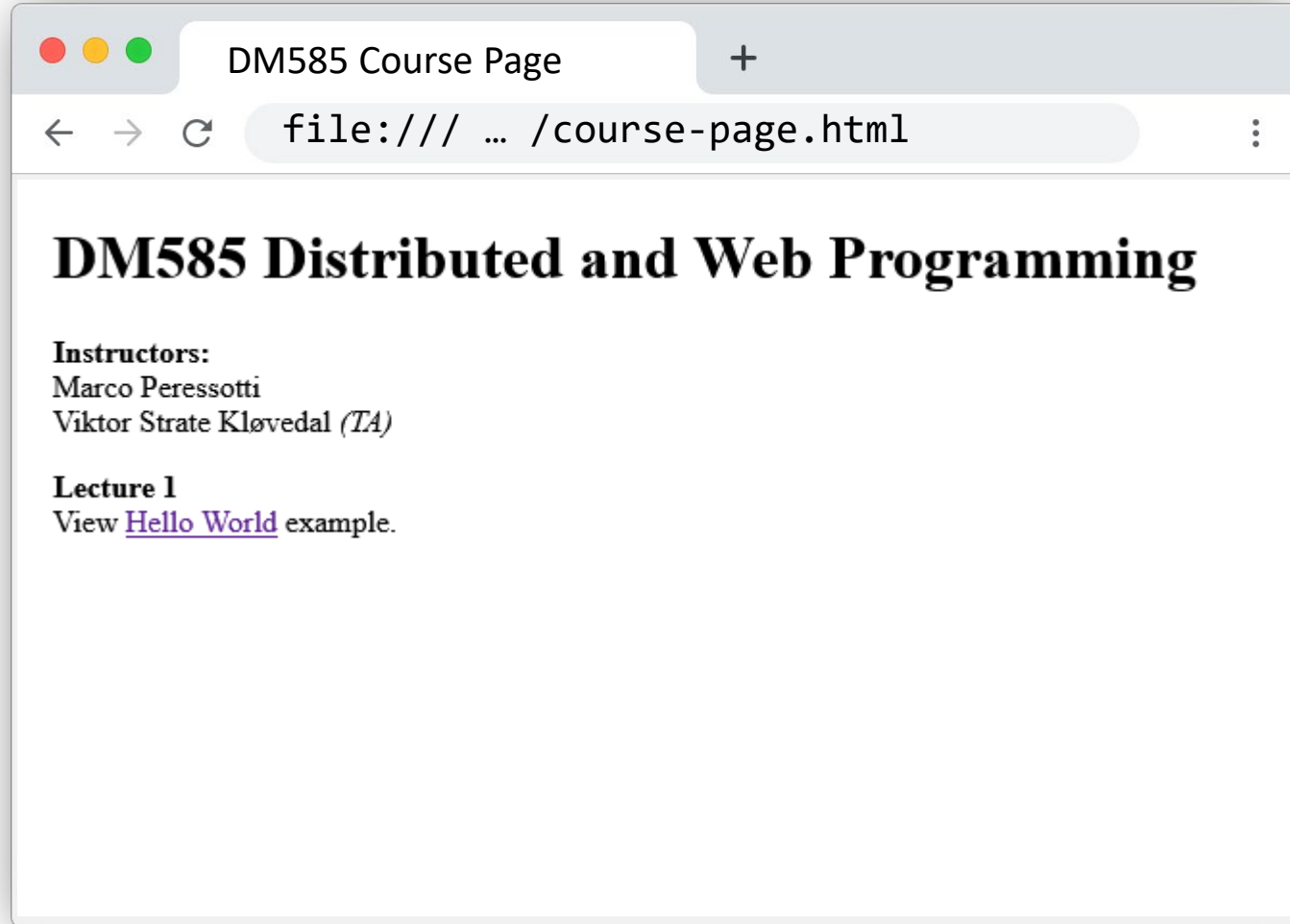
- An element usually has start and ending tags (`<p>` and `</p>`)
- The “stuff” between start and end tags is called its content
- An element can be self-closing (`img`)
- An element can have attributes (`src="html-logo.jpg"`)
- Elements can contain other elements (`p` contains `em` and `img`), forming a tree

Some HTML elements

Heading h1, h2, ... h6	<code><h1>Moby Dick</h1></code>
Paragraph	<code><p>Call me Ishmael.</p></code>
Line break	<code>since feeling is first
who pays any attention</code>
Image	<code></code>
HyperLink	<code>by H. Melville</code>
Strong (bold)	<code>Be BOLD</code>
Emphasis (italic)	<code>He's my brother and all</code>
Lists	<code>An unoreded list ... </code> <code>An ordred ordered ... </code>
List items	<code>HarpoonRope ... </code>

Exercise: course page

Let's write some HTML to make the following page:



Exercise: course page

HTML boilerplate

```
<!DOCTYPE html>
<html>
  <head>
    <title> ... </title>
  </head>

  <body>
    ...
  </body>
</html>
```

Plaintext contents of the page

DM585 Distributed and Web
Programming

Instructors:
Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1
View Hello World example.

Exercise: course page

```
<!DOCTYPE html>
<html>
  <head>
    <title>DM585 Course Page</title>
  </head>
  <body>
    <h1>DM585 Distributed and Web Programming</h1>
    <p>
      <strong>Instructors:</strong><br/>
      Marco Peressotti<br/>
      Viktor Strate Kløvedal <em>(TA)</em>
    </p>
    <p>
      <strong>Lecture 1</strong><br/>
      View <a href="hello-world.html">Hello World</a> example.
    </p>
  </body>
</html>
```

What's happening?

- We saw that HTML whitespace collapses into one space and needed **
**

```
<h1>DM585 Distributed and Web Programming</h1>  
<strong>Instructors:</strong><br/>  
Marco Peressotti<br/>  
Viktor Strate Kløvedal <em>(TA)</em>
```

- Except weirdly the **<h1>** heading was on a line of its own, and **** was not.
- 🤔
- 😄 it works... let's think about it later

Exercise: course page

```
<!DOCTYPE html>
<html>
  <head>
    <title>DM585 Course Page</title>
  </head>
  <body>
    <h1>DM585 Distributed and Web Programming</h1>
    <p>
      <strong>Instructors:</strong><br/>
      Marco Peressotti<br/>
      Viktor Strate Kløvedal <em>(TA)</em>
    </p>
    <p>
      <strong>Lecture 1</strong><br/>
      View <a href="hello-world.html">Hello World</a> example.
    </p>
  </body>
</html>
```

This path is relative to this file, so hello-world.html must be in the same folder.

Exercise: course page

```
<!DOCTYPE html>
<html>
  <head>
    <title>DM585 Course Page</title>
  </head>
  <body>
    <h1>DM585 Distributed and Web Programming</h1>
    <p>
      <strong>Instructors:</strong><br/>
      Marco Peressotti<br/>
      Viktor Strate Kløvedal <em>(TA)</em>
    </p>
    <p>
      <strong>Lecture 1</strong><br/>
      View <a href="hello-world.html">Hello World</a> example.
    </p>
  </body>
</html>
```

img src, a href, and link href can all take either **relative** or **absolute** paths to the resource:

- About
-
- <link rel="stylesheet" href="css/style.css"/>

This path is relative to this file, so hello-world.html must be in the same folder.

What is CSS?

CSS: **C**ascading **S**tyle **S**heets

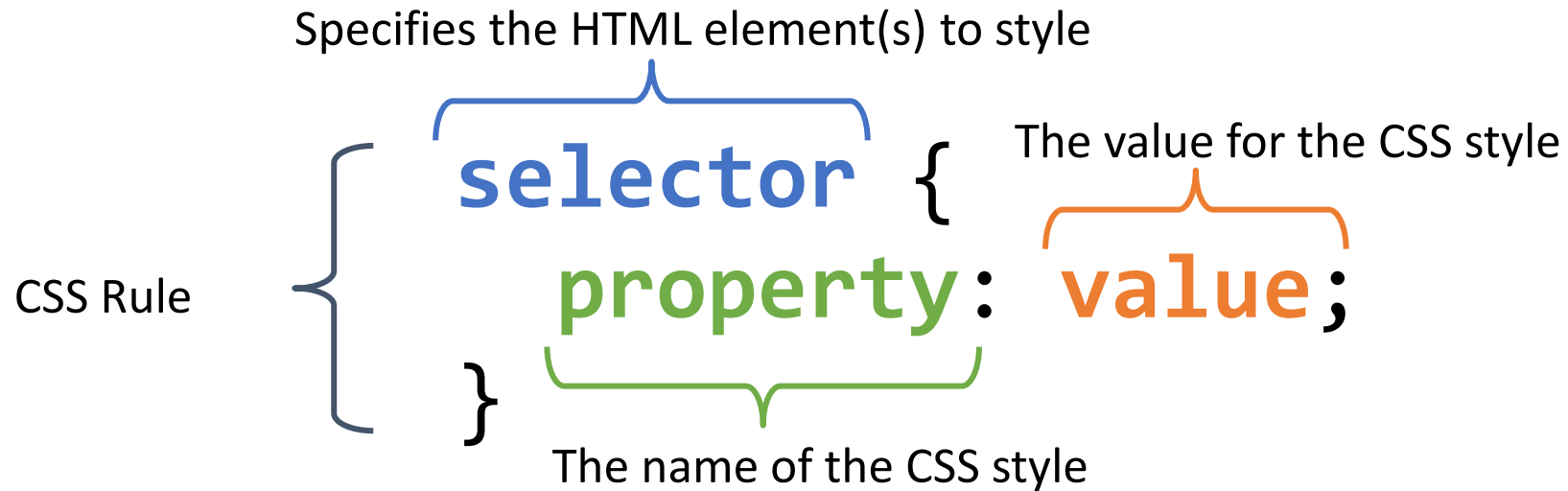
- Describes the **appearance** and **layout** of a web page
- Composed of CSS **rules**, which define sets of styles

```
selector {  
    property: value;  
}
```

What is CSS?

CSS: **C**ascading **S**tyle **S**heets

- Describes the **appearance** and **layout** of a web page
- Composed of CSS **rules**, which define sets of styles



(saved in a *filename.css* file)

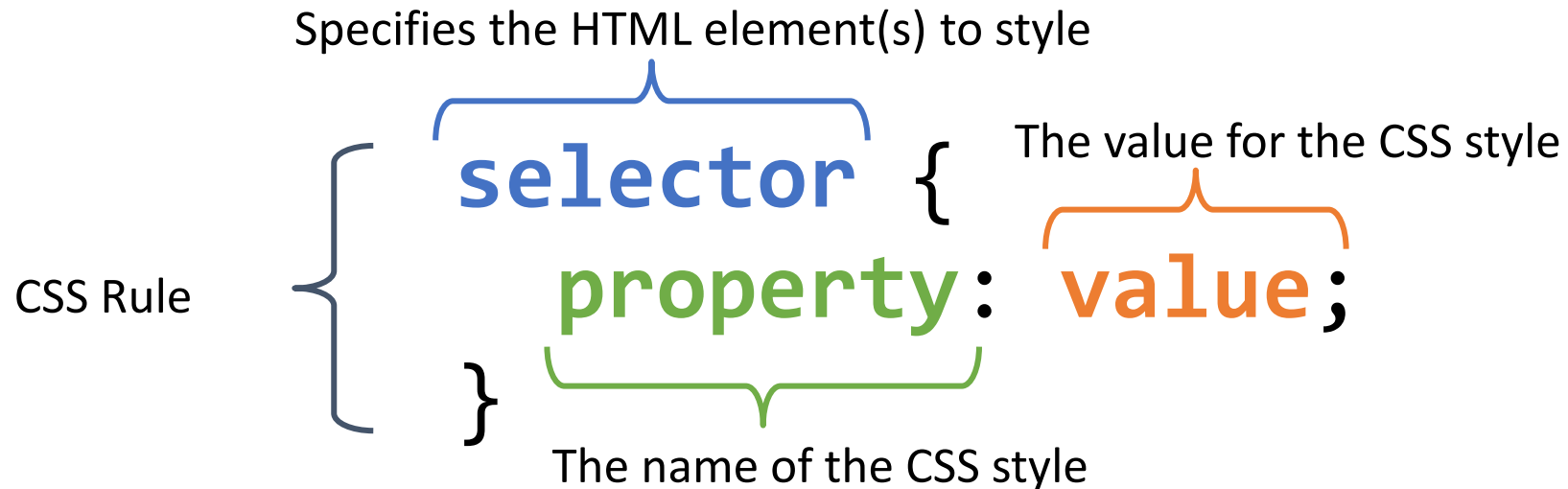
What is CSS?



I often work with something called “CCS”. Here, when I say CCS, I always mean CSS

CSS: **C**ascading **S**tyle **S**heets

- Describes the **appearance** and **layout** of a web page
- Composed of CSS **rules**, which define sets of styles



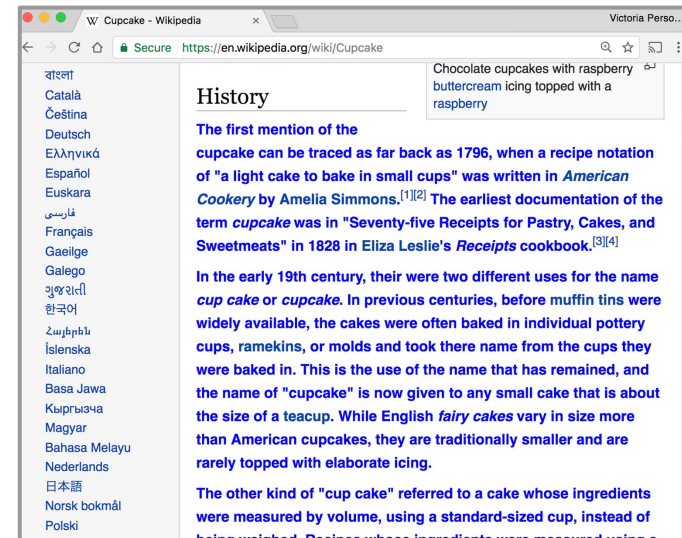
(saved in a *filename.css* file)

What is CSS?

A CSS rule specifying that

“All `<p>` elements on the page must be blue and bold”

```
p {  
  color: blue;  
  font-weight: bold;  
}
```



Linking CSS from HTML

course-page.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>DM585 Course Page</title>
    <link rel="stylesheet" href="course-page.css" />
  </head>
  <body>
    <h1>DM585 Distributed and Web Programming</h1>

    ...

  </body>
</html>
```

course-page.css

```
h1 {
  color:red;
}
p {
  color:blue;
}
```

Embedding CSS in HTML

course-page.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>DM585 Course Page</title>
    <style>
      h1 { color:red; }
      p  { color:blue; }
    </style>
  </head>
  <body>
    <h1>DM585 Distributed and Web Programming</h1>

    ...

  </body>
</html>
```

Some CSS properties

There are over [500 CSS properties](#)! Here are a few:

Font face (mdn)	font-family: Helvetica;
Font color (mdn)	color: gray;
Background color (mdn)	background-color: red;
Border (mdn)	border: 3px solid green;
Text alignment (mdn)	text-align: center;

[Mozilla Developer Network](#) (MDN) is one of the best references for HTML elements and CSS properties. The actual W3 spec is very hard to read (meant for browser developers, not web developers)

Colours in CSS

140 predefined names ([list](#))

```
color: black;
```

```
color: red;
```

```
color: green;
```

[rgb\(\)](#) and [rgba\(\)](#)

```
color: rgb(255, 0, 0);
```

```
color: rgba(255, 0, 0, 0.5);
```

[Hex values](#)

```
color: #ff0000;
```

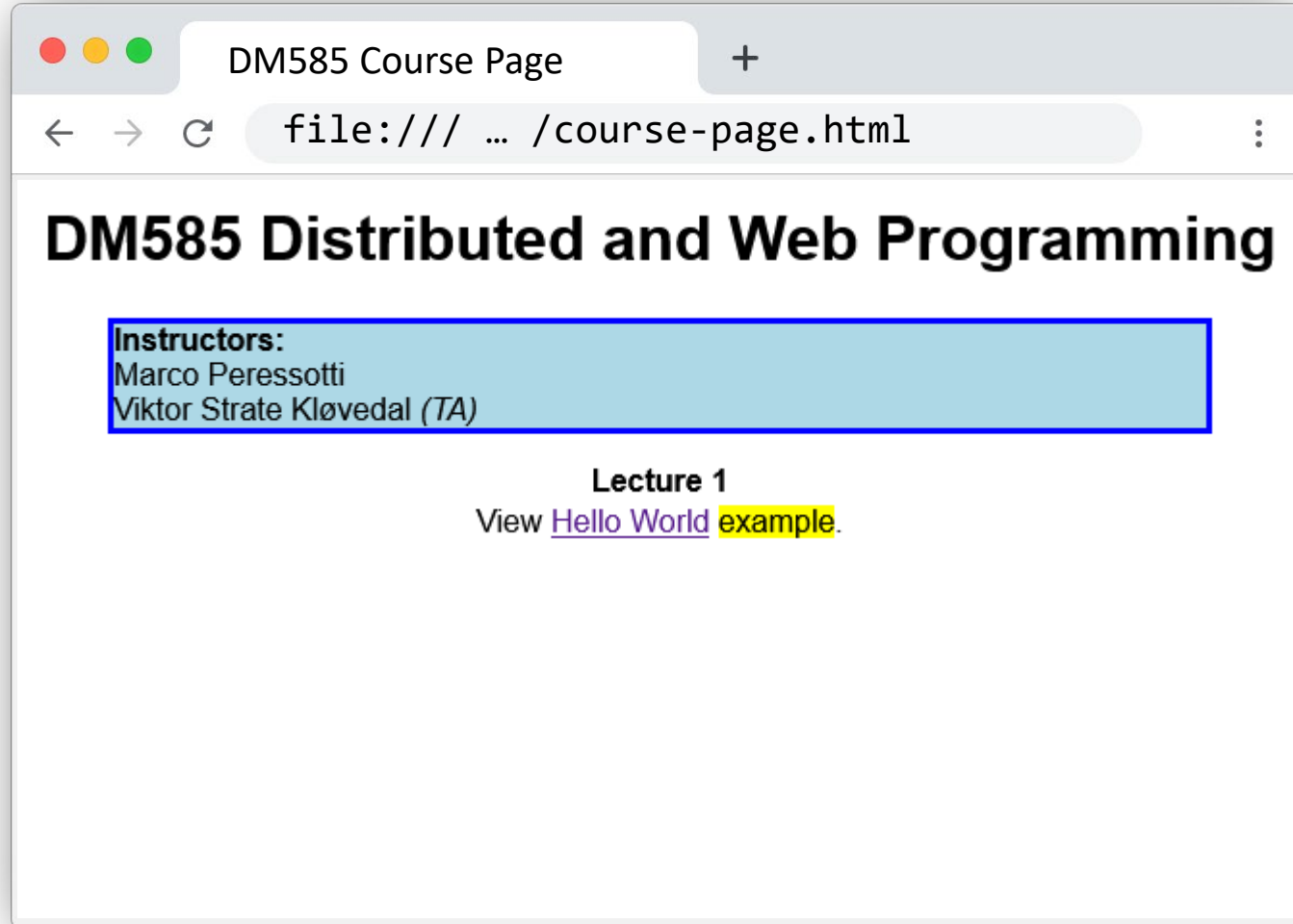
```
color: #f00;
```

```
color: #ff000080;
```

- The “a” in rgba stands for **alpha channel** and is a **transparency** value
- Hex values can be both rgb and rgba
- Hex values with repeated digits can be in compact form (**AC7** for **AACC77**)
- Prefer more descriptive/immediate options over less:
 1. Predefined name
 2. rgb / rgba
 3. Hex

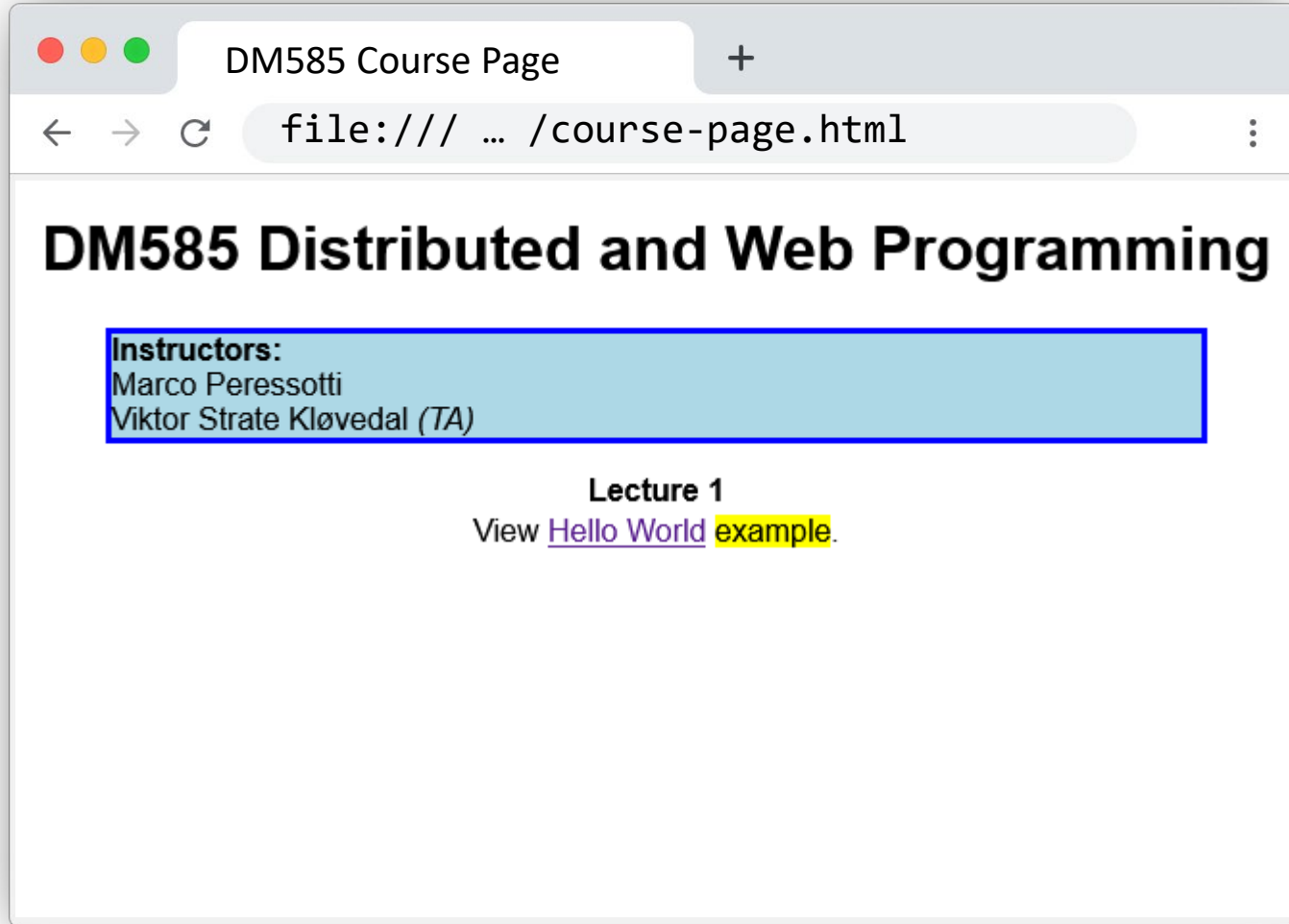
Exercise: course page

Let's write some CSS to make our page look like this:



Exercise: course page

Let's write some CSS to make our page look like this:



- Everything is in Helvetica
- The header is centred
- The “instructors” box has a
 - Border: blue solid 3px
 - Background colour: lightblue
 - Is centred.
 - Content is left-aligned
- The “lectures” block is centred
- The word “example” is highlighted in yellow.

Exercise debrief

- Some key techniques:
 - Add invisible containers in HTML to select groups of elements in CSS.
 - Apply styles to the parent/ancestor element to style the parent and all its children. (Will talk more about this later.)
- But we encountered some issues...
 - Why is HTML/CSS so bizarre??
 - There is one crucial set of rules we haven't learned yet...

Resources

- Mozilla Developers Network <https://developer.mozilla.org/en-US/>
 - HTML [docs/Web/HTML](https://developer.mozilla.org/en-US/docs/Web/HTML)
 - CSS [docs/Web/CSS](https://developer.mozilla.org/en-US/docs/Web/CSS)
 - (This is for later) JS [docs/Web/JavaScript](https://developer.mozilla.org/en-US/docs/Web/JavaScript)
 - (This is for later) HTTP [docs/Web/HTTP](https://developer.mozilla.org/en-US/docs/Web/HTTP)
- W3C (web standardisation body)
 - HTML+CSS validator <https://validator.w3.org/>
 - (Resources about HTML and CSS are mostly for a browser dev audience)